9

Лекция 6–7 апреля

Машина Тьюринга

Константин Чухарев

Дискретная математика — Весна 2027

Машина Тьюринга

> *Вычисление обычно выполняется записыванием определённых символов на бумаге.*
>
> ***— Алан Тьюринг***

Чего не хватило автомату?

Язык $a^n b^n$ нерегулярен, и это доказано дважды: леммой о накачке и теоремой Майхилла–Нерода.

Причина честная — у автомата только конечное число состояний, и бесконечный счёт в них не помещается.

> Люди считают: карандаш, бумага, стирание и переписывание.
>
> Модель вычисления обязана уметь то же самое — запоминать неограниченно много.

Человек за столом

Тьюринг в 1936 году описал вычисление так, как его делает человек с карандашом:

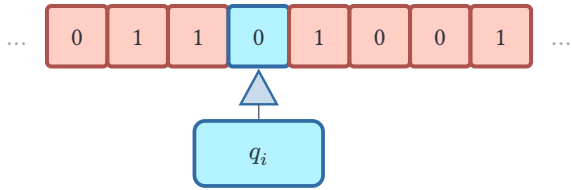
- конечное число «состояний души» — то, что человек помнит
- бумага, разбитая на клетки — то, куда он смотрит
- простые шаги — стереть, написать, сдвинуть взгляд.

Никакой магии: только символы, клетки и правила.

Всё остальное вычисление получает из их комбинаций.

Тьюринг описал вычислителя, а не вычислительную машину: под вопросом была проблема разрешения Гильберта, и чтобы доказать отсутствие алгоритма, алгоритм сначала нужно было определить.

Устройство машины



Три части с чёткими ролями: бесконечная лента — память, головка — взгляд, конечное управление — программа.

Бесконечность ленты — отсутствие предела, а не готовый склад: в каждый момент занята конечная часть, и любое вычисление касается лишь конечного числа ячеек.

Семёрка машины Тьюринга

Три части складываются в один формальный объект. Конечного в нём немного: состояния, алфавиты и таблица правил.

Определение 1 (Машина Тьюринга)

Семёрка $M = \big(Q, \Gamma, \Sigma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\big)$:

- Q — конечное множество состояний.
- Γ — алфавит ленты, содержит входной алфавит Σ и пробел.
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ — частичная функция переходов.
- q_0 — начальное состояние, q_{accept} и q_{reject} — принимающее и отвергающее.

Состояния конечны, лента — нет: весь неограниченный счёт живёт на ленте.

Компоненты: роли и границы

Компонент	Роль	Оговорка
Q	память между шагами: состояния помнят, что видели	конечна — счёт в состояниях не помещается
Γ, Σ	алфавит ленты с пробелом и входной алфавит	на входе пробелов нет, вокруг — пробелы
δ	пара (состояние, символ) даёт тройку	частична: нет правила — остановка
$q_0, q_{\text{accept}}, q_{\text{reject}}$	старт и два различимых финала	финалы различны, иначе ответ без знака

Каждая оговорка — выбор: расширения не меняют класс, а частичность δ ловит молчание.

Один шаг

Вся работа машины складывается из атомарных шагов, и шаг — это всегда срабатывание одного правила.

Определение 2 (Переход)

На паре (состояние, символ) правило δ даёт тройку (новое состояние, новый символ, сдвиг): машина записывает новый символ под головкой, сдвигает головку влево или вправо и переходит в новое состояние.

δ частична: правила для текущей пары может не быть.

Остановка вне финальных состояний по соглашению считается отверганием.

Конфигурация

Определение 3 (Конфигурация)

Мгновенное описание работы — запись uqv:

- u — содержимое ленты слева от головки
- q — текущее состояние
- v — лента от головки вправо, включая читаемую ячейку.

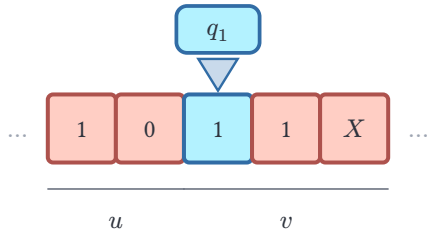
Начальная конфигурация на входе w — это q_0w: головка на первом символе.

Пример (Чтение конфигурации)

В записи $1q_0010$ имеем: $u = 1$, состояние q_0, $v = 010$.

Головка стоит на первом символе v и читает 0.

Анатомия конфигурации



На схеме лента хранит $1011X$, головка стоит на третьей ячейке, и состояние q_1 нарисовано прямо над ней. Скобки внизу показывают, какие ячейки читаются как u и как v: слева от головки и от головки вправо. Пустые ячейки за X в запись не попадают, поэтому конфигурация всегда конечна, какой бы длинной ни была занятая часть ленты.

Три исхода

Определение 4 (Исходы работы)

- МТ **принимает** вход, если достигает q_{accept}.
- МТ **отвергает**, если достигает q_{reject}.
- МТ может **зациклиться** — не достигнуть ни того, ни другого.

Третий исход — часть модели, а не каприз определения.

Программа, которая крутится вечно, должна описываться честно.

У автомата два исхода: вход читается до конца, и ответ обязан появиться.

Машина Тьюринга добавляет молчание — ответа не будет ни на каком шаге.

Что значит решить язык?

Определение 5 (Распознаваемость и разрешимость)

Язык L **распознаваем**, если есть МТ, принимающая w в точности тогда, когда $w \in L$.

На $w \notin L$ машина может отвергнуть или зациклиться.

Язык L **разрешим**, если есть МТ, которая всегда останавливается: принимает $w \in L$, отвергает $w \notin L$.

Разрешимый язык распознаваем, а обратное — нет.

Классический пример — проблема остановки.

Пример: слова, заканчивающиеся на ноль

Пример (Семёрка в деле)

Рассмотрим язык слов над $\{0, 1\}$, заканчивающихся на 0.

Состояния: q_0 — «последний символ 1», q_1 — «последний символ 0».

- $(q_0, 1) \to (q_0, 1, R)$
- $(q_0, 0) \to (q_1, 0, R)$
- $(q_0, \text{пробел}) \to (q_{\text{reject}}, \text{пробел}, R)$
- $(q_1, 0) \to (q_1, 0, R)$
- $(q_1, 1) \to (q_0, 1, R)$
- $(q_1, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$

Лента тут только возит головку: язык регулярный, память — два состояния, лента понадобится уже для подсчёта.

Полная трасса: шаг за шагом

Прогоним машину из примера выше на входе 1010: каждый шаг — одна строка.

Пример (Шаг за шагом)

Шаг	Конфигурация	Сработавшее правило
0	$q_0 1010$	$(q_0, 1) \to (q_0, 1, R)$: читаем 1, шаг вправо
1	$1 q_0 010$	$(q_0, 0) \to (q_1, 0, R)$: ноль запомнен
2	$10 q_1 10$	$(q_1, 1) \to (q_0, 1, R)$: единица сменила память
3	$101 q_0 0$	$(q_0, 0) \to (q_1, 0, R)$: снова ноль в состоянии
4	$1010 q_1$ пробел	$(q_1, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$: край — принять

Конфигурация $q_0 1010$ переходит в $1 q_0 010$ по одному правилу, и вся работа машины — цепочка таких переходов.

Трасса и разрешимость

Пример (Трасса на 10)

$$q_0 10 \to 1 q_0 0 \to 10 q_1 \text{ пробел} \to \text{принять}.$$

Последний шаг: q_1 видит пробел и принимает.

Машина читает $|w|$ символов, видит пробел и всегда останавливается.

Значит, язык разрешим — конечная память тут справляется без ленты.

Программы на ленте

> *Программы должны писаться для того, чтобы их читали люди, и лишь попутно — чтобы их исполняли машины.*
>
> ***— Гарольд Абельсон и Джеральд Сассман***

Инверсия битов

Пример (Первый алгоритм)

Заменить каждый 0 на 1 и каждую 1 на 0.

Одно состояние, три правила:

- $(q_0, 0) \to (q_0, 1, R)$
- $(q_0, 1) \to (q_0, 0, R)$
- $(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$

Эта машина вычисляет функцию, а не распознаёт язык: на ленте остаётся результат.

Разрешимость — про ответ «да или нет», вычислимость — про выход.

Правилам не нужно смотреть соседей: каждое решение принимается по паре (состояние, символ), и глобальный эффект складывается из локальных шагов.

Инкремент двоичного числа

Пример (Прибавить единицу)

Число записано на ленте, младший бит слева.

Пока читаем 1 — пишем 0 и уносим перенос вправо.

Первая 0 меняется на 1, и работа закончена.

- $(q_0, 1) \to (q_0, 0, R)$
- $(q_0, 0) \to (q_{\text{accept}}, 1, R)$
- $(q_0, \text{пробел}) \to (q_{\text{accept}}, 1, R)$

Лента — регистр неограниченной длины: новый разряд дописывается сам, и переполнения не бывает.

Инкремент: трасса

Пример (Трасса на 11)

1. $q_0 11$: читаем младшую 1, пишем 0, перенос вправо.
2. $0 q_0 1$: снова 1, пишем 0, перенос вправо.
3. $00 q_0$ пробел: пробел под головкой — пишем 1 и принимаем.

Число 3 выросло на единицу и записано как 001: младший бит снова слева.

Все единицы исчерпаны — машина дописала новый разряд.

Лента сама расширяет число: ничего «перевыделять» не нужно.

Сдвиг слова на клетку влево

Слово переезжает на клетку влево, и справа остаётся свободное место.

Пример (Перенос по одному символу)

Состояния: q_{read} читает очередной символ, q_{carry0} и q_{carry1} несут его на клетку влево, q_{hop} возвращается вправо и затирает источник.

- $(q_{\text{read}}, 0) \to (q_{\text{carry0}}, 0, L)$, $(q_{\text{read}}, 1) \to (q_{\text{carry1}}, 1, L)$
- $(q_{\text{carry0}}, \text{пробел}) \to (q_{\text{hop}}, 0, R)$, $(q_{\text{carry1}}, \text{пробел}) \to (q_{\text{hop}}, 1, R)$
- $(q_{\text{hop}}, 0) \to (q_{\text{read}}, \text{пробел}, R)$, $(q_{\text{hop}}, 1) \to (q_{\text{read}}, \text{пробел}, R)$
- $(q_{\text{read}}, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$

Клетка слева всегда свободна: у первого символа — изначально, у остальных — её освободил предыдущий шаг.

Сдвиг: трасса

Прогон на входе 10: слово сдвигается влево, справа освобождается клетка.

Пример (Слово уезжает влево)

Шаг	Конфигурация	Сработавшее правило
0	$q_{\text{read}}10$	$(q_{\text{read}}, 1) \to (q_{\text{carry1}}, 1, L)$: единица запомнена
1	q_{carry1} пробел 10	$(q_{\text{carry1}}, \text{пробел}) \to (q_{\text{hop}}, 1, R)$: слева свободно — пишем
2	$1q_{\text{hop}}10$	$(q_{\text{hop}}, 1) \to (q_{\text{read}}, \text{пробел}, R)$: источник стёрт
3	1 пробел $q_{\text{read}}0$	$(q_{\text{read}}, 0) \to (q_{\text{carry0}}, 0, L)$
4	$1q_{\text{carry0}}$ пробел 0	$(q_{\text{carry0}}, \text{пробел}) \to (q_{\text{hop}}, 0, R)$
5	$10q_{\text{hop}}0$	$(q_{\text{hop}}, 0) \to (q_{\text{read}}, \text{пробел}, R)$
6	10 пробел q_{read} пробел	$(q_{\text{read}}, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$: слово кончилось

Копирование слова

Копирование понадобится универсальной машине: она читает код и копирует его.

Пример (Пометка и разделитель)

Алфавит: $\Gamma = \{0, 1, \hat{0}, \hat{1}, Y, \text{пробел}\}$.

- q_{start} идёт вправо до пробела и ставит Y
- q_{left} возвращается к левому краю слова
- q_{pick} находит первый непомеченный символ, а на Y — конец копирования
- q_{copy0}, q_{copy1} несут символ до конца копии и дописывают его
- q_{restore} идёт влево, снимая шляпки
- q_{clean} проходит вправо и принимает на пробеле

Прочитанный символ заменяется помеченным, а копия дописывается справа за маркером Y: метка отличает прочитанное от непрочитанного, а на ленте лежат те же символы.

Копирование: лента по фазам

Вход тот же, что у сдвига, — одна единица.

Лента меняется так.

Пример (От входа до копии)

$$1 \to 1\mathrm{Y} \to \hat{1}\ \mathrm{Y} \to \hat{1}\ \mathrm{Y}1 \to 1\mathrm{Y}1$$

Маркер Y отделяет копию, шляпка помечает прочитанный символ, копия дописывается справа, чистка возвращает символам исходный вид.

Копия строится слева направо: каждый проход несёт один символ до конца копии и возвращается назад.

Время — $O(n^2)$ шагов, занятая часть ленты — $O(n)$.

Язык $a^n b^n$ на ленте

Пример (Вычёркивание пар)

Язык $a^n b^n$: конечному автомату не хватило на него состояний.

Стратегия: каждый проход вычёркивает один a слева и одну b справа.

Лента на входе aabb после первого прохода и после второго:

$$\text{aabb} \to \text{XabX} \to \text{XXXX}.$$

Когда вся лента покрыта маркерами, машина видит пробел и принимает.

Конечному автомату не хватило состояний на счёт, а машине хватает ленты.

Сравнить два счётчика — значит стереть их попарно.

Таблица переходов: вычёркивание пар

Пример (Машина для $a^n b^n$)

Алфавит $\Gamma = \{a, b, X, \text{пробел}\}$, состояния: q_0 — поиск первого невычеркнутого, q_1 — проход вправо, q_2 — поиск последней невычеркнутой b, q_3 — возврат к границе.

- q_0: $(q_0, X) \to (q_0, X, R)$, $(q_0, a) \to (q_1, X, R)$, $(q_0, b) \to \big(q_{\text{reject}}, b, R\big)$, $(q_0, \text{пробел}) \to \big(q_{\text{accept}}, \text{пробел}, R\big)$
- q_1: $(q_1, a) \to (q_1, a, R)$, $(q_1, b) \to (q_1, b, R)$, $(q_1, X) \to (q_1, X, R)$, $(q_1, \text{пробел}) \to (q_2, \text{пробел}, L)$
- q_2: $(q_2, X) \to (q_2, X, L)$, $(q_2, b) \to (q_3, X, L)$, $(q_2, a) \to \big(q_{\text{reject}}, a, L\big)$, $(q_2, \text{пробел}) \to \big(q_{\text{reject}}, \text{пробел}, R\big)$
- q_3: $(q_3, a) \to (q_3, a, L)$, $(q_3, b) \to (q_3, b, L)$, $(q_3, X) \to (q_0, X, R)$

Три ловушки закрывают все несовпадения: лишняя b ловится в q_0, a после b — в q_2 на символе, лишние a — поиском b до пробела.

Трасса: проход 1, шаги 0–4

Полная трасса на aabb, конфигурация за конфигурацией.

Пример (Вычеркнуть первый a и дойти до края)

Шаг	Конфигурация	Сработавшее правило
0	q_0 aabb	$(q_0, a) \to (q_1, X, R)$: вычеркнули первый a
1	Xq_1 abb	$(q_1, a) \to (q_1, a, R)$
2	Xa q_1 bb	$(q_1, b) \to (q_1, b, R)$
3	Xab q_1 b	$(q_1, b) \to (q_1, b, R)$
4	Xabb q_1 пробел	$(q_1, \text{пробел}) \to (q_2, \text{пробел}, L)$: край достигнут

Трасса: возврат, шаги 5–8

С конца вычёркивается последняя b, головка возвращается к границе.

Пример (Вычеркнуть последнюю b и вернуться)

Шаг	Конфигурация	Сработавшее правило
5	Xab q_2 b	$(q_2, b) \to (q_3, X, L)$: вычеркнули последнюю b
6	Xa q_3 bX	$(q_3, b) \to (q_3, b, L)$: возврат влево
7	Xq_3 abX	$(q_3, a) \to (q_3, a, L)$
8	q_3 XabX	$(q_3, X) \to (q_0, X, R)$: граница — к поиску

Трасса: проход 2, шаги 9–14

Осталось два невычеркнутых символа — один a и одна b.

Пример (Вычеркнуть оставшуюся пару)

Шаг	Конфигурация	Сработавшее правило
9	Xq_0 abX	$(q_0, a) \to (q_1, X, R)$: второй проход начался
10	XX q_1 bX	$(q_1, b) \to (q_1, b, R)$
11	XXb q_1 X	$(q_1, X) \to (q_1, X, R)$
12	XXbX q_1 пробел	$(q_1, \text{пробел}) \to (q_2, \text{пробел}, L)$
13	XXb q_2 X	$(q_2, X) \to (q_2, X, L)$: вычеркнутый хвост
14	XX q_2 bX	$(q_2, b) \to (q_3, X, L)$: последняя b вычеркнута

Трасса: приём, шаги 15–18

Пример (Приём)

Шаг	Конфигурация	Сработавшее правило
15	Xq_3 XXX	$(q_3, X) \to (q_0, X, R)$: граница — к поиску
16	XX q_0 XX	$(q_0, X) \to (q_0, X, R)$: пропуск маркеров
17	XXX q_0 X	$(q_0, X) \to (q_0, X, R)$
18	XXXX q_0 пробел	$(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$: всё вычеркнуто
19	XXXX q_{accept}	принято: $n = 2$, как и было

Каждый проход уменьшает невычеркнутую часть ровно на два символа, поэтому работа кончается: либо приём, либо одно из отвергающих правил.

Остановка гарантирована на любом входе — таблица описывает решатель.

Палиндромы

Пример (Сравнение концов)

Палиндром над $\{a, b\}$ одинаково читается в обе стороны, например abba и bab.

Стратегия: сравнить крайние символы, стереть оба и повторять, пока строка не кончится.

Состояния: q_0 читает с начала, q_a и q_b несут символ вправо, p_a и p_b сверяют его с правым краем, q_{back} возвращает головку.

При сверке три исхода: совпадение стирает правый символ, несовпадение отвергает, пробел слева — середина нечётного слова.

МТ видит оба конца строки ценой многократных проходов: запоминание через состояние — то же, что у автомата, но возвраты делает головка.

Палиндром: отвергание

Сверка кончается при первом же несовпадении, и остаток слова машина не читает.

Пример (Несовпадение на входе ab**)**

$$q_0\ \text{ab} \to X q_a\ \text{b} \to \text{Xb}\ q_a\ \text{пробел} \to X p_a\ \text{b} \to \text{Xb}\ q_{\text{reject}}\ \text{пробел}.$$

Запомненный символ несёт состояние: p_a сверяет с a, p_b — с b.

Палиндром: проход 1

Полный прогон на abba: каждый шаг — одна строка, маркер X помечает стёртую ячейку.

Пример (Запомнить a и сверить правый край)

Шаг	Конфигурация	Сработавшее правило
0	q_0 abba	$(q_0, a) \to (q_a, X, R)$: a стёрт и запомнен
1	Xq_a bba	$(q_a, b) \to (q_a, b, R)$
2	Xb q_a ba	$(q_a, b) \to (q_a, b, R)$
3	Xbb q_a a	$(q_a, a) \to (q_a, a, R)$
4	Xbba q_a пробел	$(q_a, \text{пробел}) \to (p_a, \text{пробел}, L)$: край — развернуться
5	Xbb p_a a	$(p_a, a) \to (q_{\text{back}}, X, L)$: край совпал — стереть
6	Xb q_{back} bX	$(q_{\text{back}}, b) \to (q_{\text{back}}, b, L)$: возврат влево

Палиндром: возврат и второй заход

q_{back} доходит до границы, q_0 стирает следующий символ, и машина сверяет второй край.

Пример (Стереть b и развернуться у правого края)

Шаг	Конфигурация	Сработавшее правило
7	Xq_{back} bbX	$(q_{\text{back}}, b) \to (q_{\text{back}}, b, L)$
8	q_{back} XbbX	$(q_{\text{back}}, X) \to (q_0, X, R)$: граница — к чтению
9	Xq_0 bbX	$(q_0, b) \to (q_b, X, R)$: b стёрт и запомнен
10	XX q_b bX	$(q_b, b) \to (q_b, b, R)$
11	XXb q_b X	$(q_b, X) \to (q_b, X, R)$: мимо стёртого хвоста
12	XXbX q_b пробел	$(q_b, \text{пробел}) \to (p_b, \text{пробел}, L)$: край — развернуться
13	XXb p_b X	$(p_b, X) \to (p_b, X, L)$: пропуск стёртых

Палиндром: приём

В нечётном палиндроме середину сравнивать не с чем: машина стирает её и принимает на пробеле у левого края.

Пример (Второе сравнение и приём)

Шаг	Конфигурация	Сработавшее правило
14	XX p_b bX	$(p_b, b) \to (q_{\text{back}}, X, L)$: край совпал — стереть
15	$X q_{\text{back}}$ XXX	$(q_{\text{back}}, X) \to (q_0, X, R)$: граница — к чтению
16	XX q_0 XX	$(q_0, X) \to (q_0, X, R)$: пропуск маркеров
17	XXX q_0 X	$(q_0, X) \to (q_0, X, R)$
18	XXXX q_0 пробел	$(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$: всё стёрто
19	XXXX q_{accept}	принято: abba — палиндром

Два прохода на пару дают $O(n^2)$ шагов, многоленточной машине хватает $O(n)$.

Приёмы программирования

Примеры собираются из повторяющихся приёмов.

Приём	**Идея**
Запоминание через состояние	«вижу символ a» — перейди в состояние «помню a» и неси его, пока не понадобится
Маркеры	обработанные символы заменяются на X: «уже учтено» отделяется от «ещё нет»
Проход к краю и возврат	«иди вправо, пока не пробел», шаг влево, переключение — цикл с условием на краю

Счёт на МТ — это всегда расстановка и пересчёт маркеров.

Память машины растёт не числом состояний, а занятой частью ленты.

Композиция машин

Машины собираются друг из друга: выход одного блока — вход следующего.

Определение 6 (Композиция)

Если M_1 переводит ленту из u в v, а M_2 — из v в w, состояние останова M_1 отождествляется с начальным состоянием M_2.

Из блоков собираются большие программы.

Пример (Собранная машина)

Удвоение унарного числа, затем инверсия битов: составная машина вычисляет $1^n \to 0^{2n}$.

На входе 11 удвоитель даёт 1111, инвертор — 0000.

Останов составной машины — останов последнего блока.

Машина как данные

Правило перехода — это данные: пара на входе, тройка на выходе. Таблицу переходов можно положить в обычную структуру данных.

Пример (Таблица на Rust)

```
fn step(q: &str, a: char) -> Option<(char, isize, &'static str)> {
    match (q, a) {
        ("q0", '1') => Some(('1', 1, "q0")),
        ("q0", '0') => Some(('0', 1, "q1")),
        ("q0", '#') => Some(('#', 1, "rej")),
        ("q1", '1') => Some(('1', 1, "q0")),
        ("q1", '0') => Some(('0', 1, "q1")),
        ("q1", '#') => Some(('#', 1, "acc")),
        _ => None,
    }
}
```

Интерпретатор: цикл с головкой

Пример (Печать конфигурации uqv на каждом витке)

```
fn main() {
    let mut tape: Vec<char> = "1010#".chars().collect();
    let (mut q, mut i) = ("q0", 0);
    while q != "acc" && q != "rej" {
        let u: String = tape[..i].iter().collect();
        let v: String = tape[i..].iter().collect();
        println!("{}", format!("{u} {q} {v}").trim_start());
        let Some((b, d, q2)) = step(q, tape[i]) else { break };
        tape[i] = b;
        i = (i as isize + d) as usize;
        q = q2;
    }
}
```

Что печатает симулятор?

Пример (Запуск на 1010**)**

```
q0 1010#
1 q0 010#
10 q1 10#
101 q0 0#
1010 q1 #
```

Пять строк — пять шагов таблицы переходов, и цикл вышел в acc.

Роль головки играет индекс i, а символ # изображает пробел: у вектора нет бесконечности, край помечен явно.

Машина как процессор

Приёмы, из которых собраны программы, — прообраз настоящего процессора.

> Состояние играет роль счётчика команд, маркер — флага, проход по ленте — цикла.
>
> Таблица переходов конечна, и её можно выписать строкой символов: программу удаётся положить на ленту и читать как данные.

Отсюда один шаг до машины, которая исполняет любую программу.

Универсальность и устойчивость

> *Только тьюринговская работа дала точную и несомненно адекватную дефиницию формальной системы.*
>
> ***— Курт Гёдель***

Программа как строка

Пример (Кодировка $\langle M \rangle$)

Состояния нумеруются: $q_0 = 1$, $q_{\text{accept}} = 2$, $q_{\text{reject}} = 3$.

Символы и направления — следующие числа: $0 = a$, $1 = b$, «пробел» $= c$, $L = l$, $R = r$.

Переход $\delta(q, a) = (q', b, D)$ — пятёрка чисел, таблица — пятёрки подряд.

Инкрементная машина даёт строку $\langle M \rangle = (1, b, 1, a, r)(1, a, 2, b, r)(1, c, 2, b, r)$.

Раз машина — строка, программа превращается в данные: её можно положить на ленту, скопировать, передать другой машине.

Некорректные строки по договорённости — код машины, которая на любом входе сразу отвергает, так что каждый текст описывает хоть какую-то машину.

Машины можно перенумеровать

Теорема 1 (Счётность машин)

Машины Тьюринга образуют счётное множество: они выписываются в список $M_1, M_2, M_3, \ldots$ без пропусков.

Доказательство

Докажем перечислением строк.

Строки перечисляются по возрастанию длины, внутри длины — лексикографически.

Корректные строки становятся машинами списка, некорректные — фиктивными машинами, так что пропускать нечего.

Годится любая нумерация, порождаемая алгоритмом: аргумент опирается на существование списка.

Универсальная машина

Теорема 2 (Универсальная машина)

Существует машина U, которая на входе $\langle M \rangle \sharp w$ принимает, отвергает и зацикливается ровно тогда, когда это делает M на слове w.

Доказательство

Докажем построением симулятора с тремя дорожками: программа $\langle M \rangle$, текущая лента M, состояние и маркер позиции головки.

Один шаг U: прочитать символ под маркером, найти в таблице пятёрку с этой парой, переписать ячейку, сдвинуть маркер, записать новое состояние.

U конечна и одна на все машины: интерпретаторы, виртуальные машины и эмуляторы — её инженерные потомки.

Один шаг универсальной машины

Пример (Шаг симуляции)

Пусть M — инкремент, вход 11.

На ленте U лежит $\langle M \rangle \sharp 11 \sharp 1$: программа, слово, номер текущего состояния.

U читает на дорожке работы первый символ 1, берёт состояние 1 и находит пятёрку $(1, b, 1, a, r)$: записать 0, сдвиг вправо.

Через три таких шага дорожка работы показывает 001 — ровно то, что оставляла бы сама M.

Симуляция точна: U повторяет каждый шаг M, а не только результат.

Медленнее — да, слабее — нет.

Эквивалентность вариантов

Определение содержит произвольные на вид решения: две стороны бесконечности, сдвиг ровно на клетку, один исход на шаг.

Теорема 3 (Устойчивость класса)

Односторонняя лента, сдвиг S (остаться на месте), k лент с k головками и недетерминированный выбор не расширяют класс распознаваемых языков: каждую такую машину симулирует обычная семёрка.

Определение МТ — не догадка об устройстве настоящей машины, а точка схождения: меняй удобства, класс языков прежний.

Как устроена симуляция?

Доказательство

Докажем симуляцией, вариант за вариантом.

Односторонняя лента. Правую половину кладём на верхнюю дорожку, левую — на нижнюю отражённо.

Сдвиг на месте. Обходится через свежее состояние: влево с записью b, затем вправо.

Много лент. Ленты выкладываются столбиками с маркерами головок, шаг занимает два прохода по занятой части.

Недетерминизм. Дерево конфигураций обходится в ширину, по шагу на ветвь.

«Дорожки» — не новая конструкция: ячейка хранит пару символов, алфавит-произведение конечен.

Цена симуляции

Вариант	Цена симуляции	Откуда берётся
односторонняя лента	постоянный множитель	две полуленты на двух дорожках
сдвиг S	2 шага за 1	заход влево и возврат вправо
k лент	$O(t^2)$ шагов за t шагов	занятая часть растёт на константу за шаг, и каждый шаг — проход по ней
недетерминизм	до b^t конфигураций	обход дерева в ширину: ветви длины до t

Эквивалентность говорит, что решить можно, и молчит, насколько это дорого: вопрос «можно ли без экспоненты» — главный вопрос теории сложности.

Недетерминированная машина — не многопроцессорный компьютер: ветвление — абстракция «угадал и проверь», симулятор исполняет ветви по очереди.

Тезис Чёрча–Тьюринга

Все варианты сошлись к одной машине.

Осталось спросить, измеряет ли она вычислимость вообще.

Тезис Чёрча–Тьюринга: всякая функция, вычислимая каким угодно механическим процессом с конечным описанием, вычислима на машине Тьюринга.

Статус — не теорема: доказательство связывает два формальных объекта, а здесь одна сторона, «вычислимо интуитивно», формальным объектом не является.

Статус — определение, проверяемое опытом: формализмы сошлись к одному классу, а процедура вне машинного класса за девяносто лет не предъявлена.

Свидетельства и следствие

Формализм	Автор, год	Чем вычисляет
λ-исчисление	Чёрч, 1936	подстановка термов вместо ленты
общерекурсивные функции	Гёдель–Клини, 1936	композиция и минимизация
машины Поста	Пост, 1936	продукции переработки слов
регистровые машины	Минский, 1961	арифметика над целыми регистрами
языки программирования	с 1950-х	интерпретатор сам симулируется на МТ

Рабочее следствие тезиса: неразрешимость, доказанная для МТ, закрывает задачу для всех языков и машин сразу.

Полнота конкретного языка — уже теорема, а не тезис: интерпретатор МТ пишется на этом языке, и симуляция строится явно.

Проблема остановки

> *Тестирование показывает наличие ошибок, но не может доказать их отсутствие.*
>
> ***— Эдсгер Дейкстра***

Диагональная таблица

Строки — машины M_i, столбцы — входы w_j, значения клеток взяты для примера.

Машина	**1**	**10**	**11**	**100**	**101**
M_1	+	−	«цикл»	+	−
M_2	«цикл»	+	+	«цикл»	«цикл»
M_3	+	−	+	+	−
M_4	«цикл»	+	−	−	+
M_5	−	«цикл»	+	«цикл»	+

$+$ — принял, $-$ — отверг, «цикл» — не остановился.

Диагональные клетки задают язык: w_i лежит в D, когда M_i не принимает w_i.

Инвертированная диагональ — таблица, с которой не совпадает ни одна строка.

Диагональный язык

Теорема 4 (Диагональный язык D)

Язык $D = \{w_i \mid M_i \text{ не принимает } w_i\}$ не распознаваем никакой машиной Тьюринга.

Доказательство

Докажем от противного: пусть M_k распознаёт D.

Спросим о слове w_k — входе с номером, равным номеру машины.

По определению D: $w_k \in D \leftrightarrow M_k$ не принимает w_k.

По свойству распознавателя: $w_k \in D \leftrightarrow M_k$ принимает w_k.

Собирая: M_k принимает w_k в точности когда не принимает — противоречие.

Машин счётное число, языков несчётное: диагональ предъявляет один из лишних.

Проблема остановки

Диагональ показала, что языки без распознавателя существуют.

Определение 7 (Проблема остановки)

HALT $= \{\langle M \rangle w \mid M$ останавливается на $w\}$: по коду машины и входу, записанным одной строкой, узнать, остановится ли вычисление.

HALT распознаваем: симуляция U — и есть распознаватель, она принимает, если M остановилась.

Честная оговорка: на неостанавливающемся входе симулятор молчит вечно, ответа «нет» он не даёт.

«Подождать подольше» не помогает: нет числа шагов, после которого позволено сказать «не остановится», — это свойство модели.

Неразрешимость

Теорема 5 (Тьюринг, 1936)

Проблема остановки не разрешима: не существует машины, которая на каждой паре $\langle M \rangle, w$ останавливается и верно отвечает, остановится ли M на w.

Запрещён не симулятор — симуляция работает.

Запрещён всегда останавливающийся безошибочный предсказатель остановки.

Доказательство: от противного

Доказательство

Докажем от противного: пусть решатель H всегда останавливается и верно отвечает 1 (остановится) или 0 (нет) на каждой паре $\langle M \rangle, w$.

Построим машину D: на входе $\langle M \rangle$ она запускает H на $\langle M \rangle, \langle M \rangle$ и поступает наоборот — при ответе 1 вечный цикл, при 0 останов.

Спросим о D её собственный код: что делает D на входе $\langle D \rangle$?

Случай 1. D остановился — значит, H ответил 1 и D уходит в цикл. Противоречие.

Случай 2. D не остановился — значит, H ответил 0 и D по построению останавливается.

Оба исхода невозможны, значит, H с заявленными свойствами не существует.

Оговорки и цена вопроса

Построение D — диагональ: H применяется к паре, где слева и справа один и тот же код, — шаг по диагонали таблицы.

Машина D не запрещена как таковая: она определена через H, и уничтожено лишь допущение «решатель существует».

HALT распознаваем, но не разрешим.

Дополнение $\overline{\text{HALT}}$ даже не распознаваемо.

Инженерное эхо: анализатора, который для любой программы гарантированно решает, зависнет ли она, не существует — отсюда таймауты, watchdog'и и лимиты ресурсов.

Что мы поняли?

Автомат получил ленту и стал универсальным, но универсальность немедленно показала и предел. Диагональ не распознаётся ни одной машиной, а проблема остановки распознаваема, но не разрешима.

- МТ — бесконечная лента плюс конечное управление: неограниченная память при финитном описании.
- Конфигурация uqv — полный снимок работы, вычисление — цепочка конфигураций, исходов три.
- Программы пишутся приёмами: символ в состоянии, маркеры, проходы, композиция — лента заменяет счётчик, и $a^n b^n$ разрешимо.

Программа как данные и граница машин

Машина оказалась строкой, строка — данными на ленте, и универсальная машина исполнила любую программу. На этой же идее держится главный негативный результат.

- Машина — строка $\langle M \rangle$, машины счётны, а универсальная U исполняет любую: программа стала данными.
- Варианты — односторонняя лента, k лент, недетерминизм — не меняют силу, только цену: $O(t^2)$ и b^t.
- Тезис Чёрча–Тьюринга — проверенное опытом определение вычислимости, а не теорема.
- Диагональ даёт язык без распознавателя, а HALT распознаваем, но не разрешим.

Вопросы для самопроверки

Вопросы для самопроверки — о машине Тьюринга

1. Почему $a^n b^n$ нерегулярен, но разрешим машиной Тьюринга?
2. Назовите все семь компонентов семёрки МТ и роль каждого.
3. Что записывает конфигурация uqv и как выглядит начальная конфигурация для входа 10?
4. Чем распознаваемость языка отличается от разрешимости?
5. Как машина сдвигает слово на клетку влево и почему несущее состояние всегда попадает на пробел?
6. Как устроена кодировка $\langle M \rangle$ и почему машины можно выстроить в список?
7. Что делает универсальная машина и какие три дорожки она ведёт?
8. Во сколько обходится симуляция k-ленточной машины и откуда берётся множитель?
9. Воспроизведите доказательство неразрешимости HALT: что такое H и D, где возникает противоречие?

Что почитать

- m25 — «Определение машины Тьюринга», «Примеры машин Тьюринга», «Стандартные приёмы программирования», «Устойчивость и варианты», «Тезис Чёрча–Тьюринга», «Универсальная машина Тьюринга».